Brand Sentiment Analyzer — Mentor Answers & Demo Guide

Answers to the questions asked during the build/demo cycle, consolidated in one file.

---

1. What the app is

Full-stack product **Brand Sentiment Analyzer** for a tracked brand (default `Brand`, set via `BRAND_NAME`):

```
User -> Auth & Roles -> Mention Aggregation -> Sentiment Classification -> Trend Visualization ->
Report Generation
```

- Collects mentions (simulated generator + real live sources).
- Classifies each mention: `positive / neutral / negative` (+ confidence) via ML.
- Extracts topics (universal set): `refund, delivery, support, warranty, price, quality, performance, software, apps, design, availability`.
- Shows KPIs, trends (stacked area), topics (bar), sentiment share (donut), sources, latest complaints.
- Generates CSV / PDF reports.

---

2. How the app is started (Docker)

```
cd E:\BrandSentiment
docker compose up -d
```

- Frontend: `http://localhost:8080`
- Backend API: `http://localhost:8000/docs`
- Health: `http://localhost:8000/api/health`
- Login: `admin / admin123`

Stop: `docker compose down`. Status: `docker compose ps`.

---

3. How live data is pulled (the core mechanism)

Admin → 🌍 **Pull REAL live mentions** → `POST /api/admin/ingest/real?brand=<name>&limit_each=<n>`.

Pipeline (`backend/ingestion/real_sources.py` + `ingestion/scheduler.py`):

1. **Scheduler**: APScheduler job, every `INGEST_EVERY_MINUTES` (5 min default), batch size 8; source `simulated` (built-in) or `both` (sim + live). Pausable/resumable via Admin (`/ingest/pause|resume`, `/ingest/trigger`,

`/ingest/status`, `/ingest/jobs`).

2. **Live fetchers** (each returns `(items, note)`, never raises):
   - HackerNews: Algolia search API (`hn.algolia.com`).
   - Reddit: public RSS search (`reddit.com/search.rss`).
   - Mastodon: open public timelines (`mastodon.world`, `mas.to`, ...).
   - Bluesky: public search API.
   - GDELT: world news article titles.
3. **Classify & store**: for each fetched item → `engine.predict(text)` → sentiment + confidence → `extract_topics` → insert into `mentions` table (fields: text, source, author, brand, sentiment, confidence, topics, posted_at).
4. Returns `{"added": n, "sources": [...notes], "backend": ...}`.

Manual live pull takes a `brand` string (e.g. `Samsung`, or default from `BRAND_NAME`).

---

4. How manual data gets in (CSV)

(a) Single manual mention

`POST /api/mentions` (manager/admin) — body `{"text": "..."}` → auto-classified → stored.

(b) CSV bulk upload

`POST /api/mentions/upload-csv` (multipart file) → `ingestion/csv_import.parse_csv`.

Created demo CSV → `backend/data/demo_mentions.csv`:

```
text,sentiment,author,source,posted_at,brand
"Brand delivery took forever and the box arrived damaged, furious about
this",negative,delivery_fan,reddit,2026-10-18 09:12:00,Brand
"Shoutout to Brand support, they resolved my issue in minutes",positive,helper_kim,twitter,2026-10-
18 11:02:00,Brand
...
```

- Required: `text`. Optional: `sentiment` (`positive|neutral|negative|pos|neg|neu|0|2|4`), `author`, `source`, `posted_at` (formats: `%Y-%m-%d %H:%M:%S`, `%Y-%m-%d`, `%m/%d/%Y`, ...), `brand`.
- If `sentiment` provided → **trusted** (confidence = 1.0); else **auto-classified**.
- Response: `{"added": n, "trusted_labels": n, "auto_classified": n}`.
- UI: Mentions page → "Add / import mentions" card → file input (`.csv`).

© Seed demo

Admin → `POST /api/admin/seed-demo` → 150 mentions over 30 days (`generator.generate_mentions` with product-neutral templates).

---

5. The Kaggle dataset (for retrain)

- Location expected: `backend/data/kaggle_twitter.csv` (or bundled fallback `backend/data/sample_tweets.csv`).
- Format accepted: headerless 2-col `(target, text)` or full 6-col sentiment140 `(target, ids, date, flag, user, text)` ; or headered `(label, text)` .
- Label map: `0→negative, 4→positive, 2→neutral` .
- Retrain: Admin → Retrain → upload CSV (`POST /api/admin/retrain` multipart) or use bundled dataset.
- Training: TF-IDF (1-2 grams, 15k features, sublinear) + LogisticRegression (balanced) → stratified sample ~200k rows → save to `models_out/` .
- Neutral threshold: 0.6 (below → binary low-confidence → neutral).

---

6. Features added during the session (so mentor sees current state)

- **Admin** → 🧹 **Wipe all demo data** (`POST /api/admin/reset`): wipes mentions/jobs/reports + report files, keeps users & model.
- **Theme**: dark/light toggle (🌙 / 🌞) persisted via `localStorage` , `data-theme` on `<html>` .
- **Redesign**: "Aurora Glass" dark theme; animated aurora blobs, glass cards, page-in animations, KPI stagger + count-up, skeleton shimmer, segmented day-range control, KPI icons, chart header chips, auth split-screen, admin status pill, etc.
- **Register fix**: error display normalized via `extractError` ; client-side validation (username ≥3, password ≥8, email format).

---

7. Quick demo flow for mentor

1. `docker compose up -d` → login `admin/admin123` .
2. Admin → **Wipe all demo data** → **Seed 150 demo mentions** → Dashboard fills.
3. Mentions → **Upload CSV** (`demo_mentions.csv`) → see `added / trusted_labels / auto_classified` .
4. Mentions → **Add mention** manually → auto-classify → see sentiment + topics.
5. Admin → **Pull REAL live mentions** → brand name → added rows appear (real HN/Reddit/Mastodon/Bluesky/GDELT).
6. Admin → **Retrain** (upload Kaggle CSV) → accuracy updates.
7. Reports → generate PDF/CSV.

---

8. Answers to the questions asked, in simple terms

Q asked	A
What does the app do?	It watches what people say about a brand and tells you how they feel (positive/negative/neutral), grouped by topics, with charts + reports.
How does it pull live data?	Via 5 source fetchers (HN, Reddit, Mastodon, Bluesky, GDELT) through `real_sources.ingest`, triggered by scheduler or manually.
How to configure brand?	`BRAND_NAME` in root `.env` → restart compose → data uses it; navbar stays “Brand Sentiment Analyzer”.
How is the CSV created?	Header `text,sentiment,author,source,posted_at,brand` + rows; validated through `parse_csv` (16 demo rows).
How to get Kaggle dataset?	Download Twitter Sentiment140 from Kaggle → upload via Admin → Retrain → upload CSV; or place at `backend/data/kaggle_twitter.csv`.
Is data injected dynamically or static?	Dynamic — scheduler every 5 min (batch 8) + manual triggers + CSV + live pull.
How admin adds data?	Admin: seed-demo, run-now, pull-live, pause/resume; manager: add text / upload CSV; all store → classified.
Why was register broken?	`detail` array on 422 caused React render crash; fixed with `extractError` + validation.
How to run on new PC?	Docker-only: unzip transfer, install Docker Desktop, `docker compose up --build -d`.